

1 Các bài toán lân cận, mỗi cái một đánh đố riêng (Neighbouring Tasks)

PHỤ THUỘC C/14-bai-toan-cv-loi/01-detection-hai-giai-doan và 03-segmentation (RoIAlign, heatmap, độ phân giải); B/07-chia-du-lieu-va-danh-gia (slice, metric theo nhóm); B/08-dich-chuyen-phan-phoi (long-tail).

TÓM TẮT

Mục này gom bốn họ bài toán mà kỹ sư gặp hằng ngày nhưng không nằm gọn trong trục khung/pixel: ước lượng tư thế, OCR, học biểu diễn cho truy hồi, và hai chế độ phân bố dữ liệu (fine-grained, long-tail). Điểm chung của cả bốn là quyết định quan trọng nhất không phải chọn kiến trúc mà là *chọn dạng đầu ra*: điểm, heatmap, chuỗi, hay embedding. Dạng đầu ra kéo theo luôn cả loss, metric và chế độ hỏng. Đọc xong bạn phát biểu được một bài toán thì giác lạ dưới dạng đúng ngay từ đầu, thay vì phát hiện mình chọn sai sau khi đã huấn luyện xong. Nếu chỉ nhớ một điều: chọn sai dạng đầu ra thì không kiến trúc nào cứu được, và triệu chứng của nó luôn xuất hiện ở chỗ mô hình phải biểu diễn sự *mơ hồ* — khớp bị che, ký tự nhoè, hai người rất giống nhau.

KIẾN THỨC CẦN CÓ TRƯỚC

- **dạng đầu ra** — kiểu dữ liệu mà mô hình phải sinh ra — một nhãn, một khung, một mask, một điểm, một chuỗi ký tự, hay một vector. Nó quyết định luôn hàm mất mát và thước đo, nên đây là quyết định thiết kế đắt giá nhất.
- **hồi quy (regression) và phân loại (classification)** — hồi quy là dự đoán một số thực (tọa độ x); phân loại là chọn một trong các lựa chọn rời rạc (ô lưới nào chứa khớp). Hai cách này hỏng theo kiểu rất khác nhau khi câu trả lời mơ hồ.
- **keypoint và pose** — keypoint là một điểm có ý nghĩa trên vật (khủy tay, góc biến số); pose là tập hợp các keypoint của một đối tượng cùng quan hệ giữa chúng.
- **heatmap** — một bản đồ xác suất cùng cỡ với feature map, giá trị cao ở nơi mô hình tin rằng có keypoint. Khác với một cặp tọa độ, heatmap biểu diễn được “có hai khả năng”.
- **embedding** — một vector số chiều cố định biểu diễn một ảnh, học sao cho hai ảnh cùng danh tính thì gần nhau. Nhận dạng trở thành đo khoảng cách, nên thêm một người mới không cần huấn luyện lại.
- **tập lớp mở (open-set)** — tình huống mà các lớp lúc chạy thật không có trong tập huấn luyện — lý do phân loại softmax không dùng được và metric learning ra đời.
- **long-tail** — phân bố dữ liệu mà vài lớp chiếm gần hết số mẫu còn hàng trăm lớp chỉ có vài chục mẫu; accuracy tổng bị các lớp đông thống trị nên che kín phần đuôi.
- **lát (slice)** — một tập con dữ liệu chia theo một thuộc tính có ý nghĩa — theo lớp, theo kích thước vật, theo điều kiện chụp. Báo cáo metric theo lát là cách duy nhất thấy được các chế độ khó.

Đặt vấn đề

Ba mục trước đi theo hai trục quen thuộc: nhãn ở mức khung và nhãn ở mức pixel. Nhưng phần lớn hệ thống thị giác thật lại đứng ở chỗ khác. Cánh tay robot cần *tọa độ khớp* chứ không cần khung; hệ đọc biển số cần một *chuỗi ký tự* đi qua ba khối nối tiếp; hệ nhận dạng khuôn mặt cần trả lời “có phải cùng một người” cho những người chưa từng có trong tập huấn luyện, tức đầu ra không thể là một nhãn trong tập cố định. Mục này gom bốn họ bài toán như vậy.

↪ **tiền đề** B/05 — chọn dạng đầu ra chính là bước thứ hai trong bộ ba đầu vào – đầu ra – tín hiệu giám sát; sai ở bước này thì mọi lựa chọn sau đó đều sai theo.

Chúng không nối thành một mạch lịch sử duy nhất, nhưng chúng chia sẻ một điểm chung đáng học: ở mỗi họ, quyết định quan trọng nhất là *chọn dạng đầu ra*, và chọn sai dạng đầu ra thì không kiến trúc nào cứu được.

Học xong mục này bạn làm được gì mà trước đó không làm được? Nhận một yêu cầu mơ hồ kiểu “đọc số công-tơ từ camera” hay “nhận ra người quen ở cửa”, bạn viết ngay được bộ ba đầu vào – đầu ra – tín hiệu giám sát cùng metric đúng. Kèm theo là danh sách chỗ hệ thống sẽ hỏng.

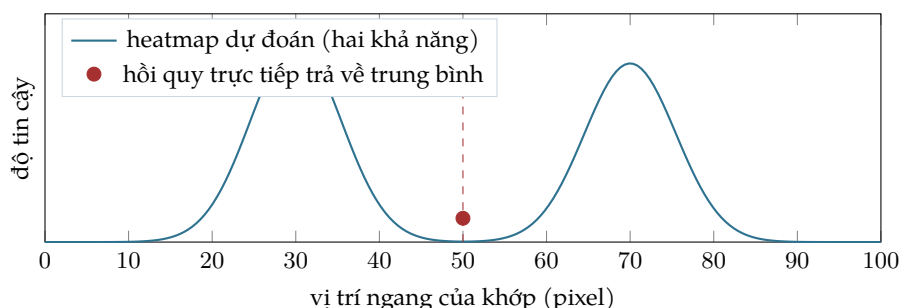
TRỰC GIÁC

Phân loại xây một cánh cửa cho mỗi lớp: muốn thêm lớp thì phải xây thêm cửa, tức huấn luyện lại. Metric learning không xây cửa mà vẽ *bản đồ*: mọi ảnh được đặt vào một không gian sao cho khoảng cách phản ánh “cùng danh tính hay không”. Thêm một người mới chỉ là ghim thêm một điểm mốc lên bản đồ đã có. Sức mạnh và điểm yếu đều nằm ở đó: bạn không phải xây lại gì cả, nhưng nếu tấm bản đồ vẽ sai thì mọi điểm mốc ghim lên nó đều sai theo, và không có cách nào sửa cục bộ.

A. Keypoint và pose: hai lần chọn dạng (Keypoint and Pose Estimation)

Ước lượng tư thế phải chọn hai lần. Lần thứ nhất là *dạng đầu ra của một khớp*, và chỉ có hai phương án.

- **Hồi quy trực tiếp hai số (x, y)** — hiển nhiên và rẻ nhất. Nhưng mạng phải nén một bản đồ tin cậy hai chiều thành một điểm, qua các tầng fully connected. Cách này không biểu diễn được sự mơ hồ: khi một khớp bị che, câu trả lời đúng là “có hai khả năng”, còn hồi quy trả về trung bình của hai khả năng — một điểm mà cả hai giả thuyết đều bác bỏ.
- **Dự đoán một bản đồ nhiệt (heatmap)** — một bản đồ xác suất cho mỗi khớp, lấy argmax làm vị trí [1]. Heatmap giữ được sự mơ hồ. Huấn luyện cũng ổn định hơn nhiều vì bài toán trở thành phân loại theo pixel. Cái giá là bộ nhớ, và một lỗi lượng tử hoá đi kèm.



Hình 1. Vì sao dạng đầu ra quyết định chế độ hỏng. Khi một khớp bị che, câu trả lời đúng là “một trong hai chỗ này”. Heatmap biểu diễn được điều đó bằng hai đỉnh, và tầng sau có thể chọn hoặc hoãn quyết định. Hồi quy trực tiếp bị hàm mất mát bình phương kéo về trung bình của hai khả năng — một điểm mà cả hai giả thuyết đều bác bỏ. Đây đúng là lý do một bộ ước lượng trạng thái không bao giờ thay một phân bố đa đỉnh bằng kỳ vọng của nó.

Hình 1 là ví dụ nhỏ nhất cho luận điểm trung tâm của mục này.

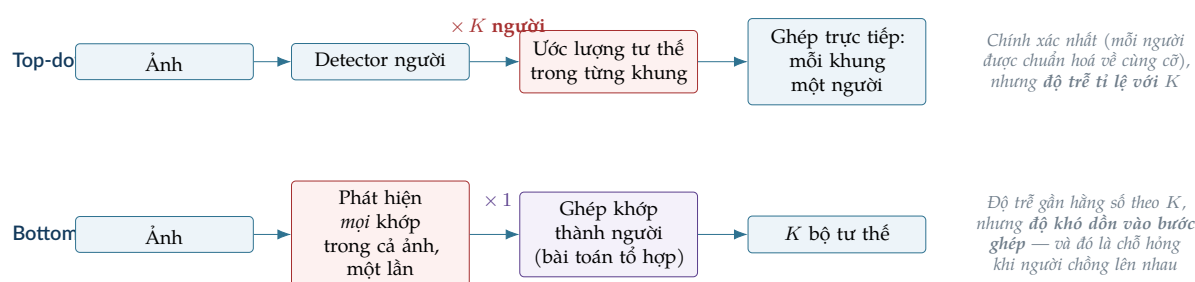
Lỗi lượng tử hoá đó tính được bằng tay. Với heatmap ở stride 4 trên ảnh 256×192 , lưới đầu ra là 64×48 ; argmax chỉ định vị được tới bội số của 4 pixel, nên sai số hệ thống là ± 2 pixel ngay cả khi mô hình hoàn hảo. Với người cao 200 pixel trong ảnh, 2 pixel là 1% — chấp nhận được; nhưng với bàn tay 40 pixel thì đó là 5%, và mọi bản triển khai nghiêm túc đều phải thêm một nhánh hồi quy độ lệch dưới pixel. Về bộ nhớ: 17 khớp $\times 64 \times 48 \times 4$ byte ≈ 209 KB cho một người — không đáng kể; nhưng ở độ phân giải 384×288 và 133 điểm (cả bàn tay và mặt) thì con số nhân lên hơn hai chục lần, và đó là lúc lựa chọn bắt đầu đau.

Lần chọn thứ hai là *cách xử lý nhiều người*. **top-down** chạy detector trước rồi ước lượng tư thế trong từng khung. Cách này chính xác nhất, vì mỗi người được chuẩn hoá về cùng kích thước.

Nhưng chi phí tỉ lệ tuyến tính với số người, nên độ trễ phụ thuộc nội dung cảnh — một tính chất tệ hại cho hệ thời gian thực. **bottom-up** phát hiện mọi khớp trong cả ảnh một lần rồi ghép chúng thành người, chẳng hạn bằng trường liên kết từng phần trong OpenPose [2]: thời gian gần như hằng số theo số người, nhưng bước ghép là một bài toán tổ hợp và nó chính là chỗ hỏng khi người chồng lên nhau.

→ cùng nguyên lý với C/15/03 — top-down và bottom-up lặp lại đúng dạng của tracking-by-detection và joint tracking: tách rời thì dễ gỡ lỗi nhưng chi phí tỉ lệ số đối tượng, gộp chung thì chi phí ổn định nhưng độ khó dồn vào một bước liên kết mờ ám.

Đây là cùng một cấu trúc đánh đổi mà bạn sẽ gặp lại ở tracking: tách rời thì dễ gỡ lỗi nhưng chi phí tỉ lệ với số đối tượng; gộp chung thì ổn định về chi phí nhưng đẩy độ khó vào một bước liên kết mờ ám.



Hình 2. Cùng một bài toán, hai chỗ đặt độ khó. Top-down mua độ chính xác bằng cách chạy lại khối nặng cho từng người, nên chi phí — và quan trọng hơn, *phương sai* của chi phí — phụ thuộc nội dung cảnh. Bottom-up cố định chi phí phát hiện nhưng phải giải một bài toán ghép tổ hợp mà không có lời giải chắc chắn. Đây đúng là cấu trúc đánh đổi sẽ trở lại ở tracking dưới tên tracking-by-detection và joint tracking.

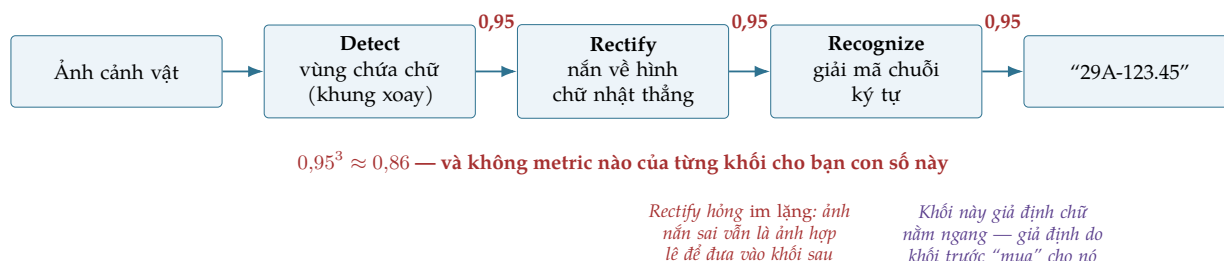
Hình 2 cho thấy vì sao lựa chọn này là lựa chọn hệ thống chứ không phải lựa chọn độ chính xác: với ngân sách mili-giây cố định, cột phải mới là cột quyết định.

B. OCR: một pipeline hoàn chỉnh trong một hệ thống nhỏ (Optical Character Recognition)

OCR đáng được mổ xẻ không phải vì nó khó nhất mà vì nó là hệ thống nhỏ nhất chứa đủ mọi thứ: phát hiện, biến đổi hình học, mô hình chuỗi, và một thước đo cấp hệ thống. Pipeline điển hình gồm ba khối nối tiếp:

1. **Detect** vùng chứa chữ. Thường là segmentation hoặc detection với khung xoay, vì chữ hiếm khi nằm ngang.
2. **Rectify** nắn vùng đó về một hình chữ nhật thẳng, bằng phép biến đổi phối cảnh hoặc một lưới biến dạng học được.
3. **Recognize** giải mã ảnh đã nắn thành chuỗi ký tự.

Điều đáng học nằm ở chỗ nối. Thứ nhất, khối rectify tồn tại vì khối recognize giả định chữ nằm ngang và đọc từ trái sang phải; đó là một giả định được đưa vào để làm khối cuối dễ đi, đúng theo nguyên lý “độ khó không biến mất, chỉ bị đẩy đi chỗ khác”. Thứ hai, lỗi của ba khối nhân lên chứ không cộng lại: nếu mỗi khối đúng 95% thì hệ thống đúng $0,95^3 \approx 86\%$, và bạn không bao giờ đọc được con số đó từ metric của từng khối. Thứ ba, thước đo đúng của OCR là tỉ lệ khớp *cả chuỗi* hoặc khoảng cách chỉnh sửa, chứ không phải độ chính xác từng ký tự — một biến số sai một ký tự là một biến số sai. Bất kỳ ai từng dựng một pipeline thì giác nhiều tầng đều nhận ra hai bài học này, và OCR là nơi dễ nhất để học chúng.

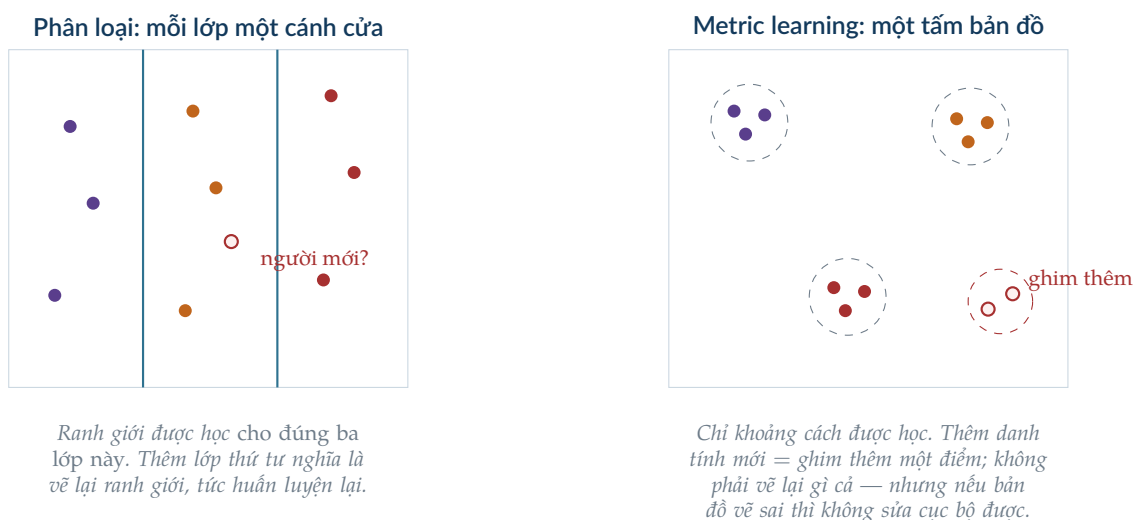


Hình 3. OCR là hệ thống nhỏ nhất chứa đủ mọi bệnh của một pipeline nhiều tầng. Độ chính xác của các khối *nhân* với nhau chứ không cộng, nên ba khối rất tốt vẫn cho một hệ thống tầm thường; khối giữa tồn tại chỉ để mua một giả định cho khối cuối; và thước đo đúng là khớp cả chuỗi chứ không phải độ chính xác từng ký tự — một biến số sai một ký tự là một biến số sai.

Hình 3 đáng nhớ vì nó áp dụng cho mọi pipeline nhiều tầng, kể cả pipeline SLAM: nếu bạn chỉ theo dõi metric của từng khối, bạn không bao giờ nhìn thấy con số mà người dùng thật sự trải nghiệm.

C. Học biểu diễn cho truy hồi (Metric Learning and Retrieval)

Nhận dạng khuôn mặt, re-identification và tìm kiếm ảnh có chung một ràng buộc mà phân loại không đáp ứng được: tập lớp *mở*. Không thể huấn luyện một softmax trên tập nhân viên của công ty rồi mỗi lần có người mới lại huấn luyện lại. Giải pháp là học một hàm *embedding* f sao cho khoảng cách trong không gian đích phản ánh quan hệ đồng nhất, rồi việc nhận dạng trở thành tìm láng giềng gần nhất. FaceNet huấn luyện bằng *triplet loss* — kéo cặp cùng danh tính lại gần hơn cặp khác danh tính ít nhất một biên α [3]; ArcFace thay bằng một biên góc cộng vào logit của softmax, vừa dễ huấn luyện hơn vừa cho ranh giới hình học rõ ràng trên mặt cầu đơn vị [4].



Hình 4. Vì sao tập lớp mở buộc phải đổi dạng đầu ra. Bên trái, mô hình học *ranh giới*, nên số lớp là một phần của kiến trúc và mọi lớp mới đòi một lần huấn luyện lại. Bên phải, mô hình chỉ học một phép đo khoảng cách, nên danh tính trở thành dữ liệu chứ không còn là tham số. Cái giá là chất lượng phụ thuộc hoàn toàn vào hình dạng của không gian nhúng — thứ không sửa được cho riêng một truy vấn.

Hình 4 giải thích luôn vì sao tầng tìm láng giềng xấp xỉ lại quan trọng đến thế: khi danh tính là dữ liệu, việc tra cứu trở thành một phần của hệ thống chứ không còn là một phép nhân ma trận.

Cái giá của cách tiếp cận này gồm ba phần. Thứ nhất, chất lượng phụ thuộc *hoàn toàn* vào không gian nhúng; không có cơ chế nào sửa được một truy vấn cụ thể mà không động vào toàn cục. Thứ hai, việc lấy mẫu quan trọng ngang hàm loss: phần lớn cặp ngẫu nhiên đã thoả biên nên không

sinh gradient, và mọi phương pháp thực dụng đều phải khai thác mẫu khó — lại là một chi tiết nằm trong code. Thứ ba, ở quy mô lớn phải thêm một tầng *approximate nearest neighbour* (ANN), và tầng đó có sai số riêng: recall của ANN đặt trần lên recall của cả hệ thống, độc lập với chất lượng embedding.

CẠM BÃY

Một khảo sát có kiểm soát đã chỉ ra rằng phần lớn “tiền bộ” công bố trong metric learning biến mất khi so sánh công bằng [5]. Công bằng ở đây nghĩa là cùng backbone, cùng cách lấy mẫu, cùng quy trình chọn siêu tham số. Bài học không chỉ dành cho metric learning: khi một lĩnh vực có nhiều bậc tự do nằm ngoài phần được viết trong bài (cách lấy mẫu, augmentation, lịch huấn luyện), bảng kết quả đo lường công sức tinh chỉnh nhiều hơn đo lường ý tưởng.

D. Fine-grained và long-tail: hai chế độ mà accuracy tổng che giấu

Hai chế độ khó cuối cùng không phải là bài toán mới mà là hai kiểu phân bố dữ liệu khiến mọi con số tổng trở nên vô nghĩa. **phân loại chi tiết** (*fine-grained classification*) là khi khác biệt giữa các lớp nhỏ hơn khác biệt bên trong một lớp: hai loài chim khác nhau ở màu một mảng lông, trong khi cùng một loài trông rất khác giữa tư thế bay và tư thế đậu. Hệ quả kỹ thuật là mô hình cần độ phân giải cao ở đúng vùng phân biệt và cần bất biến mạnh với tư thế — hai yêu cầu kéo ngược nhau. **phân bố đuôi dài** (*long-tail distribution*) là khi vài lớp chiếm phần lớn dữ liệu còn hàng trăm lớp chỉ có vài chục mẫu; accuracy tổng bị các lớp đầu thống trị nên có thể rất cao trong khi mô hình gần như không bao giờ dự đoán các lớp đuôi.

↔ cùng nguyên lý với B/08 — long-tail ở đây và dịch chuyển phân phối ở chương 8 có chung một cơ chế: phân bố lúc đánh giá không giống phân bố mà hệ thống thật sự gặp.

Điểm chung của hai chế độ là chúng *vô hình* với một con số tổng. Cách phát hiện duy nhất là chia dữ liệu theo lát và báo cáo metric trên từng lát — theo lớp, theo kích thước vật, theo điều kiện chụp. Đây chính là kỹ thuật ở B/07-chia-du-lieu-va-danh-gia, và lý do nó được nhắc lại ở đây là vì detection và segmentation có thêm một chiều lát riêng: kích thước vật. Bộ công cụ như *dbolya/tide* tách mAP thành lỗi phân loại, lỗi định vị, lỗi trùng lặp và lỗi nền [6], [7]; đọc bảng đó thường có ích hơn đọc thêm mười paper kiến trúc.

Bảng 1. Bốn họ bài toán lân cận, đọc theo trục “dạng đầu ra quyết định mọi thứ còn lại”.

Bài toán	Dạng đầu ra	Metric đúng	Hỏng khi nào
Pose	Heatmap hoặc tọa độ	PCK, OKS	Khớp bị che: hồi quy trả về trung bình của hai khả năng
OCR	Chuỗi ký tự	Khớp cả chuỗi, khoảng cách chỉnh sửa	Lỗi nhân qua ba khối; rectify hỏng im lặng
Retrieval	Embedding	Recall@ <i>k</i> , mAP truy hồi	Không gian nhúng sai thì không sửa cục bộ được; recall của ANN đặt trần
Fine-grained / long-tail	Nhãn (như phân loại)	Accuracy theo lớp và theo lát	Con số tổng bị các lớp đầu thống trị, che kín các lớp đuôi

Bảng 1 nên đọc từ cột thứ hai sang: chọn cột hai thì cột ba và cột bốn gần như bị quyết định theo, và đó chính là lý do bước chọn dạng đầu ra đáng dành thời gian hơn bước chọn backbone.

E. Giới hạn và trường hợp hỏng

Bốn họ trên chia nhau bốn chế độ hỏng, và điểm chung là cả bốn đều *im lặng*:

- **Pose — khớp bị che.** Heatmap trả một đỉnh yếu ở vị trí sai, và hệ thống hạ nguồn không phân biệt được “dự đoán yếu” với “không có”. *Cách chữa duy nhất*: truyền độ tin cậy đi cùng toạ độ, để tầng sau quyết định. Đây đúng là nguyên tắc mà bộ lọc trạng thái trong SLAM đã dùng từ lâu — không ai truyền một phép đo mà không truyền hiệp phương sai của nó.
- **OCR — nhân lỗi và thất bại im lặng.** Ba khối nhân xác suất đúng với nhau, và khối rectify hỏng vẫn sinh ra một ảnh hợp lệ để đưa vào khối nhận dạng, nên không có ngoại lệ nào được ném ra.
- **Retrieval — gallery trôi theo thời gian.** Chất lượng giảm dần mà không có tín hiệu nào lúc chạy, vì hệ thống không bao giờ biết câu trả lời đúng. *Cách phát hiện*: giữ một tập truy vấn cố định và đo lại định kỳ.
- **Fine-grained và long-tail — chính bạn.** Đo bằng một con số tổng rồi tin rằng hệ thống hoạt động. *Cách chữa*: báo cáo theo lát, không bao giờ chỉ báo cáo một số.

CẦN NHỚ

Ở mỗi bài toán lân cận, câu hỏi đầu tiên không phải “dùng kiến trúc nào” mà là “đầu ra nên có dạng gì” — điểm, heatmap, chuỗi, hay embedding. Dạng đầu ra quyết định loss, metric, chế độ hỏng và cả cách hệ thống hạ nguồn phải đối xử với sự mơ hồ. [T3]

F. Kết nối

B/05-thiet-ke-bai-toan là tiền đề khái niệm: chọn dạng đầu ra chính là bước hai trong bộ ba đầu vào – đầu ra – tín hiệu giám sát. B/07-chia-du-lieu-va-danh-gia và B/08-dich-chuyen-phan-phoi là chỗ xử lý nghiêm túc fine-grained và long-tail. C/15-thi-giac-khong-thoi-gian/03-flow-tracking-va-rang-buoc nối trực tiếp với phần top-down/bottom-up: tracking lặp lại đúng cấu trúc đánh đổi đó ở quy mô thời gian. C/18-pretraining-va-foundation giải thích vì sao không gian nhúng học từ pretraining quy mô lớn thường tốt hơn không gian học riêng cho một bài toán retrieval nhỏ.

Câu hỏi tự kiểm (trả lời không nhìn bài)

1. Vì sao hồi quy trực tiếp toạ độ khớp lại xử lý sai trường hợp khớp bị che, trong khi heatmap thì không?
2. Với heatmap stride 4, sai số lượng tử hoá là bao nhiêu pixel, và khi nào con số đó bắt đầu quan trọng?
3. Top-down chính xác hơn nhưng vì sao lại là lựa chọn tệ cho một hệ thống có ngân sách mili-giây cố định?
4. Ba khối OCR mỗi khối đúng 95%. Hệ thống đúng bao nhiêu, và vì sao metric từng khối không cho bạn biết điều đó?
5. Metric learning cho phép thêm lớp mới không cần huấn luyện lại. Cái giá của khả năng đó là gì, và nó lộ ra ở đâu khi hệ thống chạy thật?
6. Vì sao accuracy tổng che giấu cả fine-grained lẫn long-tail, và bạn thay nó bằng cách báo cáo nào?

Nguồn

- [1] K. Sun, B. Xiao, D. Liu, et al., “Deep High-Resolution Representation Learning for Human Pose Estimation,” in *CVPR*, 2019.
- [2] Z. Cao, T. Simon, S.-E. Wei, et al., “Realtime Multi-Person 2D Pose Estimation Using Part Affinity Fields,” in *CVPR*, 2017.
- [3] F. Schroff, D. Kalenichenko, and J. Philbin, “FaceNet: A Unified Embedding for Face Recognition and Clustering,” in *CVPR*, 2015.
- [4] J. Deng, J. Guo, N. Xue, et al., “ArcFace: Additive Angular Margin Loss for Deep Face Recognition,” in *CVPR*, 2019.
- [5] K. Musgrave, S. Belongie, and S.-N. Lim, “A Metric Learning Reality Check,” in *ECCV*, 2020.
- [6] D. Bolya, S. Foley, J. Hays, et al., “TIDE: A General Toolbox for Identifying Object Detection Errors,” in *ECCV*, 2020.
- [7] D. Hoiem, Y. Chodpathumwan, and Q. Dai, “Diagnosing Error in Object Detectors,” in *ECCV*, 2012.